

SOLID Principles

PrincipleCore ideaInterview keyword
S — Single ResponsibilityOne class → one reason to changeCohesion
O — Open/ClosedOpen for extension, closed for modificationPolymorphism
L — Liskov SubstitutionDerived object must safely replace base objectBehavioral subtyping
I — Interface SegregationDon't force clients to depend on methods they don't needSmall interfaces
D — Dependency InversionDepend on abstractions, not concrete classesDependency injection

A good mental model:

SOLID = designing classes so that changes don't cause a chain reaction of modifications.

1. Single Responsibility Principle — SRP

Definition

A class should have **one reason to change**.

It doesn't necessarily mean "one function."

✗ Bad design

```
class Employee {
public:
    void calculateSalary() {
        // ...
    }

    void saveToDatabase() {
        // ...
    }

    void generateReport() {
        // ...
    }
};
```

This class has multiple responsibilities:

- Salary calculation
- Database persistence
- Report generation

Therefore it has multiple reasons to change.

☑ Better

```
class Employee {
public:
    void calculateSalary() {
        // ...
    }
};

class EmployeeRepository {
public:
    void save(Employee& e) {
        // save to DB
    }
};

class ReportGenerator {
public:
    void generate(Employee& e) {
        // generate report
    }
};
```

Now each class has a focused responsibility.

Interview trick

If interviewer asks:

"Does SRP mean a class should contain only one method?"

No.

It means:

A class should have one **reason to change**, not necessarily one method.

2. Open/Closed Principle — OCP ★

This is one of the most important ones.

Definition

Software entities should be open for extension but closed for modification.

Meaning:

Adding new behavior should ideally require adding new code rather than modifying existing, stable code.

Classic bad example

Suppose we calculate area.

```
class Shape {
public:
    int type;
};

class AreaCalculator {
public:
    double calculate(Shape* shape) {
        if (shape->type == 1) {
            // circle
        }
        else if (shape->type == 2) {
            // rectangle
        }
        else if (shape->type == 3) {
            // triangle
        }

        return 0;
    }
};
```

What's wrong?

Tomorrow you add:

```
Square
Pentagon
Hexagon
...
```

You keep modifying `AreaCalculator`.

So this code violates OCP.

Applying polymorphism

Instead:

```
class Shape {
public:
    virtual double area() const = 0;

    virtual ~Shape() = default;
};
```

Then:

```
class Circle : public Shape {
    double radius;

public:
    Circle(double r) : radius(r) {}

    double area() const override {
        return 3.14 * radius * radius;
    }
};

class Rectangle : public Shape {
    double length, width;

public:
    Rectangle(double l, double w)
        : length(l), width(w) {}

    double area() const override {
        return length * width;
    }
};
```

Now:

```
class AreaCalculator {
public:
    double calculate(const Shape& shape) {
        return shape.area();
    }
};
```

Adding a new shape:

```
class Square : public Shape {
    double side;

public:
    Square(double s) : side(s) {}

    double area() const override {
        return side * side;
    }
};
```

We **extended** the system without modifying `AreaCalculator`.

That's OCP.

★ Interview question

"How does polymorphism help achieve OCP?"

Excellent answer:

Polymorphism allows us to program against an abstraction rather than concrete types. New derived classes can provide new behavior through the existing interface, so existing client code doesn't need to be modified.

That's a very interview-friendly answer.

3. Liskov Substitution Principle — LSP ★ ★ ★

This is probably the most misunderstood SOLID principle.

Definition

Objects of a derived class should be replaceable with objects of the base class **without breaking the correctness of the program**.

In simple words:

If B is a subclass of A, wherever I expect A, I should be able to use B without surprising behavior.

Classic Rectangle/Square problem

You might think:

```
Square IS-A Rectangle
```

Mathematically, yes.

But in software design, inheritance also represents **behavioral expectations**.

Suppose:

```
class Rectangle {
protected:
    int width;
    int height;

public:
    void setWidth(int w) {
        width = w;
    }

    void setHeight(int h) {
        height = h;
    }

    int area() const {
        return width * height;
    }
};
```

Now:

```
class Square : public Rectangle {
public:
    void setWidth(int w) {
        width = height = w;
    }

    void setHeight(int h) {
        width = height = h;
    }
};
```

Looks reasonable.

But now:

```
void test(Rectangle& r) {
    r.setWidth(5);
    r.setHeight(10);

    cout << r.area();
}
```

For a normal rectangle:

```
width = 5
height = 10

area = 50
```

But for **Square**:

```
setWidth(5)
→ width = 5, height = 5

setHeight(10)
→ width = 10, height = 10

area = 100
```

The behavior expected from **Rectangle** has been broken.

Therefore:

Square cannot safely substitute **Rectangle** in this design.

LSP violation.

The deeper meaning of LSP

Inheritance isn't simply:

```
B has all properties of A
```

It means:

```
B must honor the behavioral contract promised by A.
```

This is the key interview phrase:

```
LSP is about behavioral compatibility, not merely structural compatibility.
```

Another extremely important example

Consider:

```
class Bird {  
public:  
    virtual void fly() = 0;  
};
```

Then:

```
class Sparrow : public Bird {  
public:  
    void fly() override {  
        cout << "Flying";  
    }  
};
```

Fine.

But:

```
class Penguin : public Bird {  
public:  
    void fly() override {  
        throw runtime_error("Penguins can't fly");  
    }  
};
```

Now:

```
void makeBirdFly(Bird& b) {  
    b.fly();  
}
```

Calling:

```
Penguin p;  
makeBirdFly(p);
```

breaks the expectation of `Bird`.

So the abstraction is wrong.

How to fix it?

Don't make `Bird` promise that every bird can fly.

```
class Bird {  
public:  
    virtual void eat() = 0;  
};
```

Then:

```
class FlyingBird : public Bird {  
public:  
    virtual void fly() = 0;  
};
```

Now:

```
class Sparrow : public FlyingBird {
public:
    void eat() override {}

    void fly() override {
        cout << "Flying";
    }
};
```

And:

```
class Penguin : public Bird {
public:
    void eat() override {}
};
```

Much better abstraction.

★ OCP vs LSP

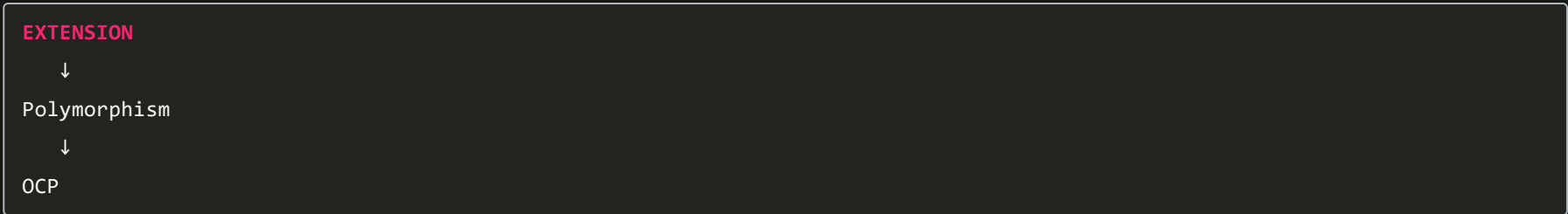
This is something interviewers love asking.

OCP

Question:

"Can I add new behavior without modifying existing code?"

Think:

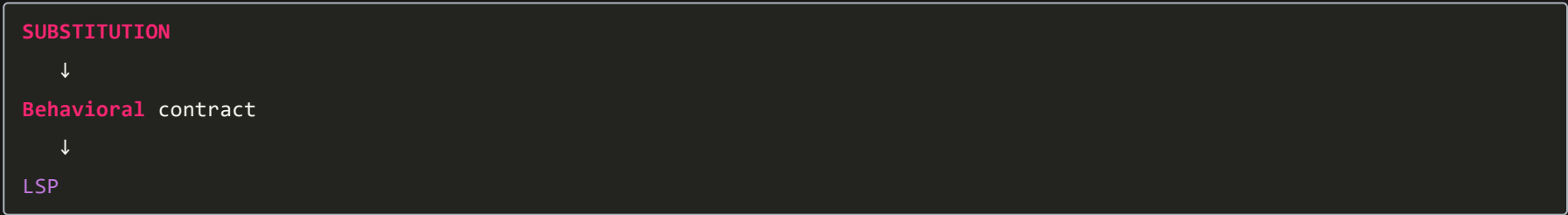


LSP

Question:

"Can a derived class be substituted for its base class without breaking behavior?"

Think:



One-line difference

OCP is about how you extend the system; LSP is about whether your inheritance relationships are valid.

4. Interface Segregation Principle — ISP

Definition

Clients should not be forced to depend on methods they don't use.

C++ doesn't have a dedicated `interface` keyword, but we commonly use abstract classes.

✗ Bad

```
class Machine {
public:
    virtual void print() = 0;
    virtual void scan() = 0;
    virtual void fax() = 0;
};
```

Now:


```
class BasicPrinter : public Machine {
public:
    void print() override {}

    void scan() override {
        // Doesn't support scanning
    }

    void fax() override {
        // Doesn't support fax
    }
};
```

We're forcing `BasicPrinter` to implement irrelevant functionality.

☒ Better

Split interfaces:

```
class Printer {
public:
    virtual void print() = 0;
};

class Scanner {
public:
    virtual void scan() = 0;
};

class Fax {
public:
    virtual void fax() = 0;
};
```

Now:

```
class BasicPrinter : public Printer {
public:
    void print() override {}
};
```

And:

```
class MultifunctionPrinter
    : public Printer,
      public Scanner,
      public Fax {

public:
    void print() override {}
    void scan() override {}
    void fax() override {}
};
```

That's ISP.

Interview keyword

Prefer many small, focused interfaces over one large interface.

5. Dependency Inversion Principle — DIP

This one is commonly confused with dependency injection.

Definition

High-level modules should not depend directly on low-level modules. Both should depend on abstractions.

And:

Abstractions should not depend on details; details should depend on abstractions.

☐ Bad

```

class MySQLDatabase {
public:
    void save() {}
};

class UserService {
    MySQLDatabase db;

public:
    void saveUser() {
        db.save();
    }
};

```

`UserService` directly depends on `MySQLDatabase`.

If tomorrow we switch to PostgreSQL:

```
MySQL → PostgreSQL
```

we have to modify `UserService`.

✓ DIP

Create abstraction:

```

class Database {
public:
    virtual void save() = 0;
    virtual ~Database() = default;
};

```

Implement it:

```

class MySQLDatabase : public Database {
public:
    void save() override {
        cout << "Saving to MySQL\n";
    }
};

```

Now:

```

class UserService {
    Database& db;

public:
    UserService(Database& database)
        : db(database) {}

    void saveUser() {
        db.save();
    }
};

```

Usage:

```

MySQLDatabase mysql;

UserService service(mysql);
service.saveUser();

```

Later:

```

class PostgreSQLDatabase : public Database {
public:
    void save() override {
        cout << "Saving to PostgreSQL\n";
    }
};

```

No modification to `UserService`.

Very common interview trap.

DIP

Design principle

Depend on abstractions rather than concrete implementations.

Dependency Injection

Technique

Give an object its dependencies from outside instead of making it create them itself.

Example:

```
UserService(Database& db)
: db(db) {}
```

That's **constructor dependency injection**.

So:

```
DIP = principle
DI  = technique commonly used to implement it
```

SOLID in one example

Imagine you're building a payment system.

Bad:

```
class Payment {
public:
    void pay(string type) {

        if (type == "UPI") {
            // UPI
        }
        else if (type == "CARD") {
            // Card
        }
        else if (type == "PAYPAL") {
            // Paypal
        }

        // save to DB
        // generate invoice
        // send email
    }
};
```

Lots of problems.

SRP

Split:

```
PaymentProcessor
InvoiceGenerator
EmailService
PaymentRepository
```

OCP

Instead of:

```
if (type == "UPI")
else if (type == "CARD")
```

use:

```
class PaymentMethod {
public:
    virtual void pay(double amount) = 0;
    virtual ~PaymentMethod() = default;
};
```

Then:

```
class UPI : public PaymentMethod {
public:
    void pay(double amount) override {}
};

class Card : public PaymentMethod {
public:
    void pay(double amount) override {}
};
```

New payment method → add new class.

LSP

Every `PaymentMethod` must honor the contract of `pay()`.

A derived payment method shouldn't unexpectedly violate the assumptions made by clients.

ISP

Don't create:

```
PaymentMethod {
    pay()
    refund()
    generateInvoice()
    sendEmail()
    ...
}
```

Split unrelated capabilities.

DIP

Payment service should depend on:

```
PaymentMethod
```

rather than:

```
UPI
```

or:

```
Card
```

Interview Questions You Should Practice

Basic

Q1. What are SOLID principles?

Know all five and their one-line definitions.

Q2. Does SRP mean one class should have only one method?

No.

One **reason to change**, not one method.

Q3. Explain OCP with a C++ example.

Use:

```
Shape
↓
Circle
Rectangle
Square
```

and explain how polymorphism avoids modifying `AreaCalculator`.

Q4. How does polymorphism help OCP?

Because new derived implementations can be added without modifying code that operates on the base abstraction.

Q5. What is LSP?

Derived objects must be substitutable for base objects without changing the correctness/expected behavior of the program.

Q6. Explain the Rectangle-Square problem.

Very important.

The issue isn't that mathematically a square isn't a rectangle.

The issue is that the **behavioral contract** of the mutable `Rectangle` abstraction doesn't hold for `Square`.

Q7. Is "is-a" relationship enough to justify inheritance?

No.

This is a great interview answer:

Inheritance should satisfy both the conceptual relationship and the behavioral contract expected by the base class. A simple "is-a" relationship isn't sufficient for LSP.

Q8. How do you detect an LSP violation?

Look for derived classes that:

```
throw unexpected exceptions
ignore base-class methods
return fundamentally different behavior
strengthen preconditions
weaken postconditions
require special checks from clients
```

Especially this smell:

```
if (dynamic_cast<Derived*>(base)) {
    // special handling
}
```

If client code constantly needs to know the derived type, your abstraction may be broken.



Very common interview scenario

Interviewer:

"You have a `Bird` base class and `Penguin` can't fly. What would you do?"

Bad:

```
class Bird {
public:
    virtual void fly() = 0;
};
```

Then Penguin throws.

Better:

```
class Bird {
public:
    virtual void eat() = 0;
};

class FlyingBird : public Bird {
public:
    virtual void fly() = 0;
};
```

Then:

```
Bird
├─ Penguin
└─ FlyingBird
   └─ Sparrow
```

This simultaneously demonstrates:

SOLID Cheat Sheet

Remember this:

```
S → One reason to change
O → Extend, don't modify
L → Derived must behave like base
I → Small focused interfaces
D → Depend on abstractions
```

Or think:

```
S → Responsibility
O → Extension
L → Substitution
I → Interfaces
D → Dependencies
```

Most important for your C++ interview

I'd prioritize them like this:

LSP ≈ OCP > DIP > SRP > ISP

Not because the others are unimportant, but because **OCP/LSP/DIP** give you the most opportunities to demonstrate actual C++ OOP **understanding** through inheritance, virtual functions, abstract classes, polymorphism, and dependency injection.

And given what you've already covered—**virtual functions, abstract classes, inheritance, `override`, `final`, constructors/destructors, smart pointers, Rule of 5**—SOLID is basically where all those C++ OOP concepts start coming together.